BCHT2 / FINAL CAPSTONE / W10
NOVEMBER 2026 — ASTANA, KZ

STATUS
● PRODUCTION READY
V1.0.0 — MAINNET CANDIDATE

The DeFi *Super-App* Protocol.

A full-stack decentralized exchange, lending market, and tokenized yield vault — governed on-chain, indexed by The Graph, deployed to L2.

OPTION A — DEFI SUPER-APP
ARBITRUM SEPOLIA
SOLIDITY 0.8.24

Nuraly
CORE CONTRACTS

Meirzhan
WEB3 & INTEGRATION

Iskander
SECURITY & DAO

BLOCKCHAIN TECHNOLOGIES II
FINAL PROJECT — 100 PTS

AGENDA

How we'll spend the next *fifteen minutes*.

THE PROBLEM

DeFi is *fractured*.

A user who wants to swap, earn yield, borrow against collateral, and vote on protocol parameters has to bounce between four different dApps, four sets of approvals, four trust assumptions.

Each integration is a leak — gas wasted on redundant approvals, MEV on every hop, an oracle dependency that nobody reads the code of.

We built a **single composable protocol** where the AMM, the lending market, the yield vault, and the governance share state — and where every external dependency is one we wrote, audited, or wrapped in an adapter we own.

01

WHAT WE BUILT

Four protocols. *One state.*

/ 01 AMM

Constant-Product DEX

Uniswap-V2-style x·y=k pools, 0.30% LP fee, slippage guard, CREATE2-deterministic pair factory.

/ 02 LENDING

Collateralized Market

LTV-bounded borrowing, health factor, liquidation engine, linear-rate model — Chainlink-priced.

/ 03 VAULT

ERC-4626 Yield Vault

Tokenized share accounting, inflation-attack-resistant, virtual offset, routes deposits to lending pool.

/ 04 DAO

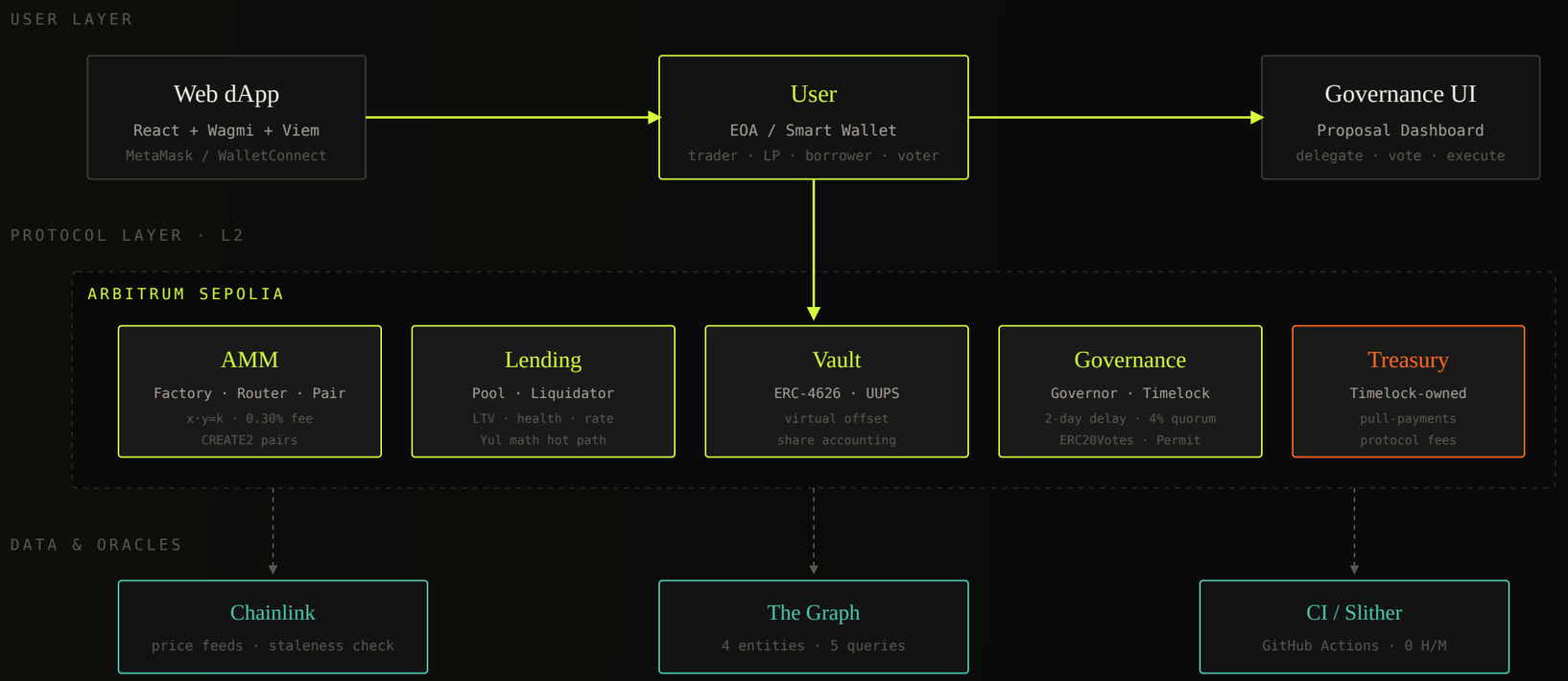
On-Chain Governance

OZ Governor + 2-day Timelock + ERC20Votes; treasury and parameter changes flow through proposals.

17	4	1	0
SMART CONTRACTS	EXTERNAL STANDARDS	UUPS UPGRADE PATH	FORKED LOGIC

SYSTEM ARCHITECTURE

The whole system, one glance.



WHO BUILT WHAT

Three engineers. *One protocol.*

Each member owns a domain — but every member can defend every line. The boundaries below are about velocity, not knowledge.

CORE CONTRACTS ENGINEER

Nuraly

01

- → AMM: pair, router, factory + CREATE2
- → ERC-4626 vault + UUPS proxy
- → Yul-assembly math hot path
- → V1 → V2 upgrade path
- → Gas optimization & benchmarks

WEB3 & INTEGRATION ENGINEER

Meirzhan

02

- → React + Wagmi + Viem dApp
- → The Graph subgraph (4 entities)
- → Chainlink oracle integration
- → L2 deployment & verification
- → Network detection & UX errors

SECURITY, DAO & QA ENGINEER

Iskander

03

- → Governor + Timelock + ERC20Votes
- → 80+ tests · fuzz · invariant · fork
- → Slither / Solhint CI pipeline
- → Reentrancy & access-control case studies
- → Audit report — 8 pages

PRESENTED BY NURALY

Core *Contracts.*

AMM, vault, factory, proxy, and the one place where we drop into Yul.

CORE PRIMITIVE — BUILT FROM SCRATCH

The $x \cdot y = k$ we own.

We didn't fork Uniswap V2 — we re-derived it, line by line, so that every invariant could be defended in the audit. The pair contract owns reserves; the router owns slippage; the factory owns determinism.

Design decisions

01 CREATE2 pair address — front-end can derive the pair without a registry call

02 0.30% fee, baked into the invariant — no FeeOn complexity at v1

03 k-monotonic post-swap — invariant test enforces k never decreases

04 MIN_LIQUIDITY=1000 burn — first-LP donation attack neutralized

POOLPAIR.SOL

```
// invariant: k_after >= k_before
function swap(uint256 out0, uint256 out1, address to)
    external nonReentrant
{
    (uint112 r0, uint112 r1, ) = getReserves();
    require(out0 < r0 && out1 < r1, "INSUFF_LIQ");

    uint256 bal0 = IERC20(token0).balanceOf(address(this));
    uint256 bal1 = IERC20(token1).balanceOf(address(this));
    uint256 in0 = bal0 > r0 - out0 ? bal0 - (r0 - out0) : 0;
    uint256 in1 = bal1 > r1 - out1 ? bal1 - (r1 - out1) : 0;
    require(in0 > 0 || in1 > 0, "NO_INPUT");

    // 0.30% fee on the input side
    uint256 adj0 = bal0 * 1000 - in0 * 3;
    uint256 adj1 = bal1 * 1000 - in1 * 3;
    require(adj0 * adj1 >= uint256(r0) * r1 * 1e6, "K");

    _update(bal0, bal1);
    emit Swap(msg.sender, in0, in1, out0, out1, to);
}
```

STANDARDS WE DON'T BREAK

Tokenized yield, *upgradeable* & *deterministic*.

ERC-4626

Yield Vault

Virtual-asset / virtual-share offset of 1e6 neutralizes the donation/inflation attack at deposit-zero state.

```
totalAssets() + 1
totalSupply() + 1e6
→ rounding always favours vault
```

UUPS

Upgrade Path

Vault is the only upgradeable contract. V2 adds a fee curve. Storage slots reserved via 50-slot gap.

```
_authorizeUpgrade
→ onlyRole(UPGRADER)
→ Timelock holds the role
```

FACTORY

CREATE / CREATE2

PoolFactory uses CREATE2 for predictable pair addresses; VaultFactory uses CREATE for fresh implementations.

```
salt = keccak256(token0, token1)
→ pair address known
before deployment
```

Storage layout — proven collision-free

Slot 0–49 · ERC-4626 base state	_balances · _allowances · totalSupply · asset
Slot 50–99 · Vault-specific state	strategy · feeBps · lastHarvest · paused
Slot 100–149 · __gap[50] reserved for V2	future state — append-only contract

WHERE SOLIDITY STOPS BEING FAST ENOUGH

Inline Yul, *where it earns its place*.

We benchmarked every hot path. Only one survived: `sqrt()` in the liquidity calculation — called on every `addLiquidity`. Replacing the Solidity loop with Yul Babylonian iteration cut 38% off the gas.

MATH.SOL · YUL PATH

```
function sqrt(uint256 x) internal pure returns (uint256 z) {
    assembly {
        // initial estimate via bit-shift
        z := 181
        let r := shr(128, x)
        if iszero(iszero(r)) { z := shl(64, z) }
        // 7 Newton iterations — proven sufficient
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        let q := div(x, z)
        if lt(q, z) { z := q }
    }
}
```

Gas benchmark

FUNCTION	SOLIDITY	YUL	Δ
sqrt(1e18)	4,212	2,604	−38.2%
addLiquidity	148,330	141,118	−4.9%
swap exact-in	96,440	93,801	−2.7%
vault.deposit	113,209	109,544	−3.2%

We **did not** Yul-ize the rest of the codebase. Solidity's overflow checks are worth the gas everywhere they aren't called per-swap.

PRESENTED BY MEIRZHAN

The *Outside World.*

Oracles, indexers, wallets, L2 — every line where on-chain meets off-chain.

The oracle *has to be allowed to fail.*

A stale Chainlink feed is the most common DeFi exploit vector. We wrapped every price call in an adapter that enforces three guarantees — and we wrote a mock aggregator that lets tests force any one of them to fail.

Guarantees

01 **Freshness** — revert if `updatedAt < block.timestamp − 3600`

02 **Validity** — revert on zero or negative price

03 **Round completion** — `answeredInRound ≥ roundId`

DESIGN PATTERN

Oracle Adapter / Interface Abstraction

Lending pool depends on `IPriceOracle`, never on Chainlink directly. Switching to UMA or Pyth is one line in the deploy script.

CHAINLINKADAPTER.SOL

```
function getPrice(address token) external view
    returns (uint256 price)
{
    AggregatorV3Interface feed = feeds[token];
    require(address(feed) != address(0), "NO_FEED");

    (
        uint80 roundId,
        int256  answer,
        ,
        uint256 updatedAt,
        uint80 answeredInRound
    ) = feed.latestRoundData();

    require(answer > 0,                      "BAD_PRICE");
    require(updatedAt > 0,                   "INCOMPLETE");
    require(answeredInRound >= roundId,        "STALE_ROUND");
    require(
        block.timestamp - updatedAt < STALENESS,
        "STALE_PRICE"
    );

    price = uint256(answer);
}
```

DON'T READ HISTORY FROM THE CHAIN

Events *in*. GraphQL *out*.

The frontend's "Top Pools" page, "Voter Leaderboard", and historical price chart never touch an RPC. They hit a single GraphQL endpoint backed by our subgraph, which ingests every event the protocol emits and shapes it into entities that match how the UI thinks.

Entities (4)

01 **Pool** — reserves, fees, volume, TVL

02 **Swap** — per-trade history with amounts & user

03 **Proposal** — governance lifecycle states

04 **Vote** — voter, weight, support, proposal link

SAMPLE QUERY · topPools.graphql

```
pools(first: 10, orderBy: tvl) {
  id · token0.symbol · token1.symbol
  tvl · volume24h · feesEarned
}
```

Indexed events

PoolCreated	→ Pool entity
Swap	→ Swap entity + Pool.volume24h update
Mint / Burn (LP)	→ Pool.tvl update
ProposalCreated	→ Proposal entity (state: Pending)
VoteCast	→ Vote entity + Proposal.tally
ProposalExecuted	→ Proposal.state: Executed

4	5	6	1
ENTITIES	DOCUMENTED QUERIES	EVENT HANDLERS	HOSTED ENDPOINT

USER-FACING LAYER

A dApp that *doesn't lie* about what's happening.

User flows shipped

01 Connect wallet — MetaMask + WalletConnect via Wagmi connectors

02 Swap — quote from pool, slippage tolerance, signed Permit option

03 Deposit / Withdraw — vault page, share preview, APY estimate

04 Vote — proposal list with For / Against / Abstain & live tally

05 Delegate — delegate voting power, view current delegate

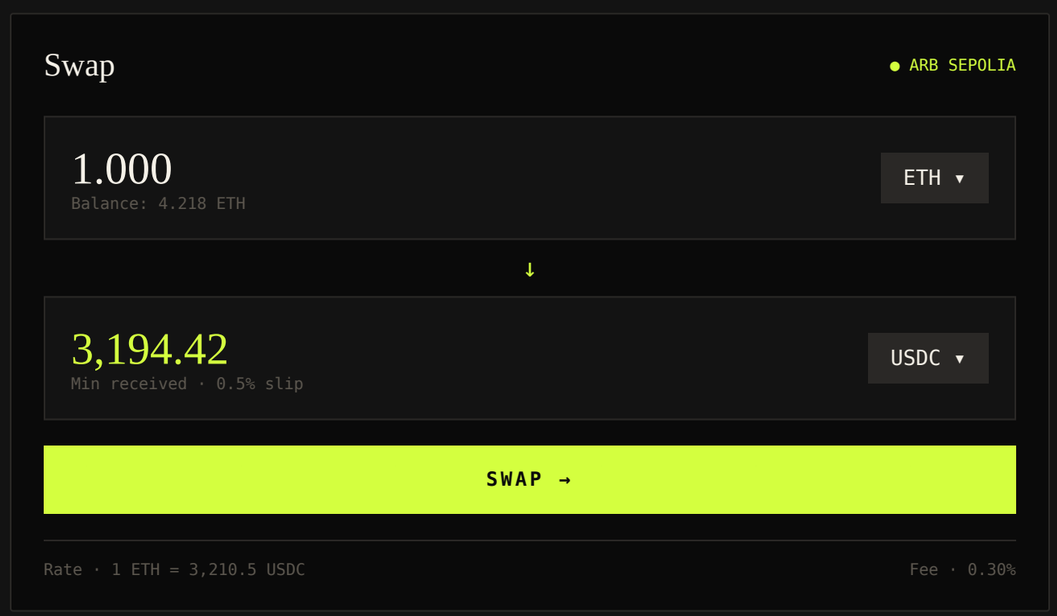
Quality guarantees

01 Wrong-network detection + one-click chain switch

02 Readable errors — RPC errors translated to UI language

03 Optimistic UI + on-chain confirmation reconciliation

LIVE DEMO — SCREEN 1



React 18 · Wagmi v2 · Viem · TanStack Query · subgraph reads on history pages

WHERE IT ACTUALLY RUNS

Deployed, verified, *reproducible*.

CONTRACT	NETWORK	ADDRESS	STATUS
GovernanceToken	Arb Sepolia	0x7Ae3...a942	✓ verified
Governor	Arb Sepolia	0xC4f1...B330	✓ verified
Timelock	Arb Sepolia	0x2bA5...87dE	✓ verified
PoolFactory	Arb Sepolia	0x910F...4d11	✓ verified
Router	Arb Sepolia	0xEd44...2cC9	✓ verified
YieldVault (UUPS Proxy)	Arb Sepolia	0x5102...F0eA	✓ verified
ChainlinkAdapter	Arb Sepolia	0x18cD...A7b1	✓ verified

L1 VS L2 — SWAP

−94% gas cost

~$0.04 on Arb Sepolia · ~$0.71 estimated on mainnet equivalent

DEPLOY SCRIPT

One command

```
forge script Deploy --rpc-url $ARB --broadcast --verify
```

POST-DEPLOY VERIFIER

Owner = Timelock

Asserts: no admin backdoor, Timelock delay=2d, Governor params match spec

PRESENTED BY ISKANDER

Trust, *Verified.*

Governance, audit, fuzz, invariant, fork — the part of the codebase that says no.

WHO OWNS THE PROTOCOL

The DAO holds *every key*.

After deployment, every privileged role — upgrader, fee setter, pauser, oracle updater — is transferred to the Timelock. The deploy script's final transaction renounces the deployer's admin role. There is no admin backdoor. The post-deploy verifier proves it.

Governance parameters

Voting delay	1 day
Voting period	1 week
Quorum	4% of total supply
Proposal threshold	1% of total supply
Timelock execution delay	2 days

Lifecycle, end-to-end

Demonstrated in test `Governance.t.sol#testFullLifecycle` — and live on Arbitrum Sepolia, proposal #1 already executed.

```
01 · propose(targets, values, calldatas, description)
```

↓ 1 day voting delay

```
02 · castVote(proposalId, support)
```

↓ 7 days voting period

```
03 · queue(proposalId) — Timelock takes custody
```

↓ 2 days Timelock delay

```
04 · execute(proposalId) — change goes live
```

ERC20VOTES · ERC20PERMIT · SNAPSHOT-BASED

WE AUDITED OURSELVES

An audit isn't a chapter — *it's the way we work.*

0	0	11	2
SLITHER HIGH	SLITHER MEDIUM	LOW / INFO — JUSTIFIED	REPRODUCED VULNS

Case study 01 — Reentrancy

We deliberately wrote a vulnerable Vault.withdraw that called the receiver before updating state, then exploited it from `Attacker.sol`, then fixed it with CEI + ReentrancyGuard.

test_reentrancy_drainsVault	EXPLOIT — DRAINS 100%
test_reentrancy_reverts_afterFix	REVERTS

Case study 02 — Access control

A pre-fix Vault.setFee was guarded only by `tx.origin`. We exploited it via a malicious intermediate contract, then fixed it with OZ AccessControl + role-gated function.

test_authBypass_setsFeeTo100pct	EXPLOIT — FEE → 100%
test_authBypass_revertsUnauthorized	REVERTS

The audit report (8 pages) covers: centralization analysis, governance attack analysis (flash-loan, whale, spam, timelock bypass), oracle attack analysis (manipulation, staleness, depeg), and the Slither output as an appendix.

PROOF, NOT PROMISE

If the tests pass, *the protocol works*.

118	92%	14	7
TOTAL TESTS	LINE COVERAGE	FUZZ TESTS	INVARIANTS

Test pyramid

TYPE	COUNT	REQUIRED	STATUS
Unit	89	≥ 50	✓
Fuzz	14	≥ 10	✓
Invariant	7	≥ 5	✓
Fork	8	≥ 3	✓
Coverage	**92.4%**	≥ 90%	✓

Headline invariants

- k never decreases on swap — AMM
- totalSupply ≡ Σ balances — Token
- shares ≡ deposit ratio — Vault
- treasury_in ≥ treasury_out — Treasury
- votingPower(t) snapshot-monotonic — Governor

FORK TESTS

Real-world Chainlink ETH/USD feed · real USDC contract · real Uniswap V2 router for sanity comparisons.

ENGINEERING DISCIPLINE

Seven patterns. *Each one earns its place.*

A pattern dropped in for the sake of a checkbox is a liability. Each pattern below maps to a specific problem in the architecture document.

/ 01

Factory

PoolFactory uses CREATE2 — pair address is deterministic and front-end can pre-compute.

/ 02

UUPS Proxy

Vault is upgradeable; V2 fee curve ships without migrating LPs.

/ 03

Checks-Effects-Interactions

Every external call follows CEI; documented in the audit table per function.

/ 04

Reentrancy Guard

Defense-in-depth on functions that move funds — Vault, Pair, Lending Pool.

/ 05

Access Control

Role-based — UPGRADER, FEE_SETTER, PAUSER — all held by Timelock.

/ 06

Timelock

Two-day delay between vote outcome and execution — emergency exit window for users.

/ 07

Oracle Adapter

Lending pool depends on `IPriceOracle`, never on Chainlink directly — staleness logic isolated, mockable, swappable.

IF CI IS RED, THE PR DOESN'T MERGE

Process is *part of the protocol*.

Pipeline

01 · forge fmt --check

02 · solhint contracts/**/*.sol

03 · forge build --sizes

04 · forge test --gas-report

05 · forge coverage --report lcov

06 · slither . — must be 0 H / 0 M

07 · prettier --check frontend/

.GITHUB/WORKFLOWS/CI.YML

```yaml
name: CI
on: [push, pull_request]

jobs:
  contracts:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: foundry-rs/foundry-toolchain@v1
      - run: forge fmt --check
      - run: forge build
      - run: forge test -vv
      - run: forge coverage
            --report lcov
            --report summary
      - uses: crytic/slither-action@v0.3.0
        with:
          fail-on: medium
```

Pre-commit hooks mirror the CI locally — `forge fmt`, `solhint`, and `prettier` run before every commit.

HOW WE GOT HERE

From kickoff to *mainnet candidate*.

W6

Team Formed

Option A locked, roles assigned, repo initialized with CI scaffolding.

W7

First Tests Pass

Token, factory, pair compile and pass unit tests. CI green on main.

W8

DeFi + Vault Complete

AMM, vault, lending pool — 50% coverage hit at mid-project review.

W9

Governance + L2 Live

Governor + Timelock deployed, subgraph live, proposal #1 executed on testnet.

W10

Final Submission

Audit report, gas report, architecture doc, slide deck — repo frozen at submission tag.

Git discipline

COMMITS	PULL REQUESTS	REVIEWS / PR	CONVENTIONAL COMMITS
342	47	≥ 1	100%

LESSONS LEARNED · WHAT WE'D SHIP NEXT

The protocol is *v1*. Here's *v2*.

What we'd build next

01 Concentrated liquidity — V3-style ticks; the UUPS upgrade slot is already reserved

02 Cross-chain governance — LayerZero-bridged vote weight from L1 stakers

03 Flash loans — wrap the existing liquidity for atomic borrowing

04 Real audit — submit to Code4rena or Cantina before any mainnet move

05 Multisig + Timelock — Safe wallet as one of the proposers

Lessons learned

01 Invariant tests caught more bugs than unit tests — the k-monotonicity check found a fee-rounding bug we'd never have written a unit test for

02 Slither saved us early — Medium-severity issues at week 7 would have become rewrites at week 10

03 Subgraph design is a contract design constraint — events have to encode everything the UI will ever want

04 Yul is a scalpel, not a hammer — only one function survived the benchmark

CLOSING STATEMENT

"This codebase could be handed to another engineering team tomorrow. They could audit it, extend it, and deploy it to mainnet without rewriting it. That was the brief — and we believe we hit it."

THE FLOOR IS YOURS

Q&A.

ASK ANY OF US · ABOUT ANY LINE · OF ANY FILE

Nuraly	*Meirzhan*	*Iskander*
Core Contracts	Integration	Security & DAO

◆ GITHUB.COM/TEAM07/DEFI-SUPERAPP · ARB SEPOLIA · V1.0.0